COMPTIA PENTEST+ · DOMAIN 5 — POST-EXPLOITATION

Establish & Maintain Persistence

Initial access is fragile — sessions expire, services restart, defenders clean up. Persistence embeds access so deeply into a target that you can return without re-running the original exploit. On the exam this whole domain rewards one instinct: matching a scenario's *OS, privilege level, network shape, and stealth need* to the right mechanism.

Maps to Objective 5.1 — "Given a scenario, perform tasks to establish and maintain persistence after gaining initial access." The twelve techniques below are the official sub-bullets, in objective order.

Knowledge cutoff caveat: verify the exact objective wording against CompTIA's current PT0-003 objectives PDF — phrasing gets revised between prints, though the technique list here matches the published bullets.

01 Scheduled tasks / cron jobs	02 Service creation	03 Reverse shell
04 Bind shell	05 Add new accounts	06 Obtain valid credentials
07 Registry keys	08 C2 frameworks	09 Backdoor: web shell / Trojan
10 Rootkit	11 Browser extensions	12 Tampering security controls

THE STEALTH SPECTRUM (MEMORIZE THE ORDERING — IT SETTLES MANY QUESTIONS)

MOST OBVIOUS	DETECTABLE	VERY STEALTHY	INVISIBLE
New accounts (in user lists), bind shells (open listening port)	Scheduled tasks, registry Run keys, services — Autoruns finds them	Valid credentials, encrypted-HTTPS C2 — looks like normal activity	Rootkit — hides its own files, processes & connections from the OS

---

00 What persistence is (and why it matters)

Persistence is continued access that survives reboots, credential resets, patching, and cleanup. The mechanism embeds a re-entry method into the environment so access can be re-established automatically — no repeat of the original exploit.

The reporting angle matters as much as the technical one: proving an attacker can hold *long-term, undetected* access communicates far greater risk to a client than proving initial access

alone. Keep one distinction sharp throughout — the **persistence mechanism** (what re-establishes access) is separate from the **access channel** (how you actually use it). A scheduled task is the mechanism; the reverse shell it launches is the channel.

◈ EXAM TELL

Every 5.1 question is really asking you to filter on four variables at once: **OS, privilege level, network constraints, and required stealth/duration.** Read the scenario for those four before you look at the answers.

---

01 Scheduled tasks & cron jobs

These abuse the OS's own automation so the operating system becomes the delivery mechanism for your payload — no attacker process has to be running, and execution looks like ordinary administrative automation.

WINDOWS — TASK SCHEDULER / SCHTASKS

`schtasks` creates, queries, and deletes tasks from the command line. A task can run at a set time, on an interval, at startup, or at logon — and crucially it can run under any user context, **including SYSTEM**. A malicious task named something like `WindowsUpdateHelper` pointing at a payload in a writable directory re-establishes a C2 connection on schedule and survives reboots because the OS executes it.

LINUX — CRON

Cron is time-based. System-wide jobs live in `/etc/crontab` and `/etc/cron.d/` (plus hourly/daily dirs) and need root; a per-user job via `crontab -e` needs only user access. A reverse-shell script fired every few minutes means a dropped connection auto-reconnects.

◈ EXAM TELL

If a scenario says the tester **loses access after a reboot**, the intended answer is almost always a scheduled task or cron job — because these are executed *by the OS itself*. "Survives reboot" + "automatic re-execution" is the fingerprint. If it also demands **elevated** re-execution on Windows, that's the SYSTEM-context scheduled task specifically.

schtasks /create → Windows · **crontab -e** → Linux user-level · **/etc/crontab** → Linux system-wide · **SYSTEM privileges** → high-value task · "survives reboot"

CLEANUP

`schtasks /delete` the task; remove the crontab entry. Document path + trigger before removing.

---

02 Service creation

Services are long-running background processes that start at boot and often run privileged — an ideal hiding place. Creating or modifying one gives a persistent, privileged foothold that blends into normal system operation.

WINDOWS — SCM / SC

Services are managed by the Service Control Manager and registered in the registry under `HKLM\SYSTEM\CurrentControlSet\Services`. `sc create` points a service's binary path at your payload. Because services commonly run as **SYSTEM**, a malicious service is both re-entry *and* privilege escalation. Display names like "Network Diagnostics" blend into the list.

LINUX — DAEMONS

Managed by `systemd` (unit files in `/etc/systemd/system/`, enabled with `systemctl enable`) or legacy `/etc/init.d/` scripts with `/etc/rc*.d/` links.

Modify vs. create: replacing a legitimate service's binary, or editing its executable path in the registry, is even stealthier — the name and description stay unchanged.

◈ EXAM TELL

When a question wants **boot-time persistence with elevated privileges**, service creation is the strong answer — it out-competes scheduled tasks specifically on the "runs before user login, as SYSTEM" axis. Watch for the phrase "long-running background process" as a direct pointer to a service/daemon.

sc create → Windows service | **systemd unit file** → Linux | **/etc/init.d/** → legacy Linux | SYSTEM account execution | service binary path → payload

CLEANUP

`sc delete` the service (Windows); `systemctl disable` + remove the unit file (Linux).

---

03 Reverse shell

A reverse shell inverts the normal client/server model: the **compromised target initiates an outbound connection back to the attacker**. You set up a listener, deliver and execute a payload, and the target dials home — giving you an interactive shell in the payload's user context.

WHY OUTBOUND WINS

Firewalls and NAT typically block unsolicited *inbound* connections but allow *outbound* ones, because users and services need to reach the internet. The reverse shell rides that asymmetry — an outbound connection that most perimeters permit without inspection. Use port 443/80 and it looks like normal web traffic; wrap it in TLS/HTTPS and IDS has little to grab.

IMPLEMENTATIONS

Bash one-liners, Python socket subprocesses, PowerShell (.NET networking), Netcat, and Meterpreter (encrypted, feature-rich C2-grade shell).

On its own, a reverse shell is not persistent — drop the connection or reboot and it's gone. It *becomes* persistence when a scheduled task, service, or registry Run key re-launches it.

◈ EXAM TELL

Any scenario about **bypassing NAT or an inbound-blocking firewall** resolves to a reverse shell. The one-line discriminator to lock in: **reverse = target connects OUT to attacker.** If a stem stresses that the shell alone didn't survive a reboot, they're testing the mechanism-vs-channel split — pair it with #01/#02/#07.

target connects out | bypasses inbound firewall | requires attacker listener | **Netcat / Meterpreter** | port 443/80 to blend

---

04 Bind shell

The mirror image: the payload **opens a listening port on the target** and waits for the **attacker to connect inbound**. Directionality is the whole point — bind = attacker connects *to* target.

WHEN IT'S USEFUL — AND ITS WEAKNESS

Bind shells fit when you have direct network access to the target (e.g., internal segments with no NAT in the way) or when *your* box can't accept inbound connections to host a listener. Their fatal limitation: they require inbound connectivity, so perimeter firewalls and NAT block them — making them impractical for most external tests. An open listening port is also easier for defenders to spot on a scan.

> ◈ EXAM TELL
>
> The single most common 5.1 trap is swapping bind and reverse. Bank the rule cold: **bind = inbound (attacker → target), reverse = outbound (target → attacker).** "Behind NAT/firewall" ⇒ reverse; "direct internal access, can't host a listener" ⇒ bind. Expect a stem that hands you specific network constraints and asks which shell fits.

attacker connects to target | listening port on target | blocked by firewall/NAT | **nc -l -p** listener | viable on internal segment

---

05 Add new accounts

Creating a user account gives a legitimate-looking access path that leans on the system's own authentication — so it survives patching and tool removal. Only deliberate account auditing removes it.

WINDOWS & LINUX

`net user /add` (or PowerShell) creates a local account; a local admin account = persistent admin access. A **domain** account (needs domain admin to create) reaches many systems across the enterprise. On Linux, `useradd` / `adduser`, then add to `sudo` / `sudoers` — and drop an attacker key into `~/.ssh/authorized_keys` for passwordless SSH that survives even a password change.

HIDING THE ACCOUNT

Windows: set the account to `0` under `...Winlogon\SpecialAccounts\UserList` to hide it from the login screen while it still works for remote access. Linux: UID < 1000 hides it from many login managers, and a `/sbin/nologin` shell makes it look like a service account while still allowing key-based SSH. Name it to mimic a service account (`svchost_svc`, `backup_usr`) or the org's naming convention.

◈ EXAM TELL

"Persistence that **survives remediation / patching / tool removal**" points at account-based persistence (and at #06 valid credentials). If the stem adds "reaches multiple systems across the enterprise," it wants a **domain** account specifically. On the stealth spectrum, note new accounts sit near the *obvious* end — they show up in account lists unless hidden.

`net user /add → Windows` `useradd / adduser → Linux` `SSH authorized_keys`

`domain account → broad reach` `survives patching`

CLEANUP

`net user [name] /delete` (Windows) or remove the Linux account + its authorized_keys entry.

---

06 Obtain valid account credentials

Instead of creating an account, take an existing one. Access via valid credentials is indistinguishable from legitimate user activity — no new account, no malicious tool, nothing to detect. It's the **stealthiest persistence available**.

WHERE CREDENTIALS COME FROM

Dumping: `Mimikatz` pulls plaintext (in some configs), NTLM hashes, and Kerberos tickets from **LSASS** memory; the **SAM** database holds local hashes. **Keylogging** captures credentials as typed (software or hardware). **Phishing** harvests them via fake login portals. And because password reuse is rampant, one captured set often unlocks many systems, VPNs, and cloud services.

USE WITHOUT CRACKING

Pass-the-hash authenticates with an NTLM hash directly; **pass-the-ticket** reuses a Kerberos ticket — no plaintext needed.

Credentials are unique among techniques: no artifact, no process, no file. As long as they stay valid, access is available.

◈ EXAM TELL

If a question asks for the **stealthiest** persistence — or one with "no artifact to detect" — the answer is valid credentials, not a rootkit. (Rootkit wins "most invisible *implant*"; credentials win "no implant at all.") **LSASS + Mimikatz** together always point at credential dumping; **pass-the-hash** is the "reuse without cracking" giveaway. Only a *password reset* revokes this access.

`LSASS → dump target` `Mimikatz → primary tool` `SAM → local hashes` `pass-the-hash / pass-the-ticket`

`stealthiest persistence`

---

07 Registry keys

Certain registry keys are read and acted on automatically at startup or logon, making them reliable triggers for attacker code on Windows.

RUN / RUNONCE KEYS

- `HKLM\...\CurrentVersion\Run` — all users at startup; **requires admin**.
- `HKCU\...\CurrentVersion\Run` — current user at logon; **user-level only**.
- `...\RunOnce` — executes once then deletes itself; good for staged payloads (HKLM & HKCU variants).

Other spots: `Winlogon`'s `Userinit`/`Shell` values (append your exe for logon persistence), and the `Services` hive (see #02).

HKCU VS HKLM — THE KEY DISTINCTION

HKCU = standard-user privileges, affects only that user. **HKLM** = needs admin, affects all users, system-wide. With only user access, HKCU Run keys give persistence *without* escalating first.

Defenders enumerate these with **Autoruns** (Sysinternals), so registry persistence is detectable with monitoring — which is why it's often paired with obfuscation.

> ◈ EXAM TELL
>
> "Windows persistence **without admin privileges**" ⇒ **HKCU Run key**. "Affects all users / system-wide" ⇒ HKLM (admin required). Know that **Run** re-fires every logon while **RunOnce** fires once and self-deletes. If the stem mentions **Autoruns**, it's cueing the *defender's* detection of exactly this mechanism.

HKCU Run → no admin · **HKLM Run** → admin, all users · RunOnce → runs once · Winlogon → logon persistence · **Autoruns** → detection

> CLEANUP
>
> Delete the specific value; restore any modified Winlogon value to its documented original.

---

08 Command & control (C2) frameworks

C2 is the most operationally capable option: a **server** (operator console) plus **agents/implants** on compromised hosts. It's not one shell — it's centralized management of many hosts, complex post-exploitation, and persistence at scale.

HOW AGENTS PERSIST

Agents keep a persistent *outbound* connection (usually HTTPS, to mimic web traffic), reconnecting on an interval. The agent itself is dropped via one of the other mechanisms — a scheduled task, service, or Run key relaunches it.

KNOW THESE THREE FRAMEWORKS

- **Empire** — PowerShell/Python; runs **in memory** to dodge file-based detection; encrypted HTTP/HTTPS.
- **Covenant** — .NET; agents are called **"Grunts"**; web UI; HTTP/HTTPS/SMB.
- **Mythic** — **modular / plugin** architecture; many agent types and channels.

Channels: HTTPS (blends in), DNS (covert), SMB (internal host-to-host). Traffic is encrypted; agents favor memory over disk.

◈ EXAM TELL

"Manage / maintain access across **multiple compromised hosts** simultaneously" ⇒ C2 framework. The three names map to one attribute each: **Empire = in-memory PowerShell/Python**, **Covenant = .NET Grunts**, **Mythic = modular**. If a stem says "avoids disk-based detection," it's pointing at in-memory execution (Empire).

`Empire → in-memory PS/Python` `Covenant → .NET, Grunts` `Mythic → modular` `agent/implant + listener` `HTTPS / DNS / SMB channels`

---

09 Backdoors — web shell & Trojan

A backdoor bypasses normal authentication to let you re-enter without re-exploiting. Objective 5.1 names three backdoor types — web shell and Trojan here; rootkit gets its own bullet (#10).

9A · WEB SHELL

A script (PHP, ASP/ASPX, JSP, Python) dropped into a web-accessible directory that executes commands sent as HTTP parameters and returns output in the response. It rides the web server's existing exposure — **port 80/443 is already open, no new port/service/process** — and its traffic looks like normal requests. Delivered via file-upload flaws, RFI, SQLi file-writes, or directory traversal with write access. Ranges from `<?php system($_GET['cmd']); ?>` to full toolkits like China Chopper. Detection needs web-log analysis or file-integrity monitoring for new files in web dirs.

`PHP/ASP/JSP in web dir` `HTTP-based command exec` `uses port 80/443` `file-upload delivery` `no new ports`

9B · TROJAN

Malware disguised as legitimate software; delivery depends entirely on **social engineering** — the user must run it. Often a real, working app with malicious code bolted on (more convincing). Once run, it installs a backdoor / C2 / persistence (via registry, scheduled task, or service) and may disable security tools. The Trojan is the *delivery*; the persistence is whatever it plants. In pentests it tests user awareness, endpoint detection, and email filtering.

`disguised as legit software` `user must execute` `social-engineering delivery` `fake update / malicious attachment`

◈ EXAM TELL

"HTTP/HTTPS command execution via a file in a web directory" and "**no new ports required**" ⇒ web shell. "Disguised as legitimate software / user must run it" ⇒ Trojan. Don't conflate the two: web shell exploits a *server* flaw; Trojan exploits a *person*.

---

10 Rootkit

The most deeply embedded persistence. A kernel-level rootkit runs at the OS's own privilege, intercepts **system calls**, and rewrites the answers — ask "what processes are running?" and it removes itself before replying. It hides not just itself but any file, process, connection, or registry entry tied to the attacker.

TYPES

- **Kernel-mode** — malicious modules/drivers; highest privilege, hardest to detect.
- **User-mode** — hooks/replaces libraries (DLLs / shared objects); weaker but easier and more stable.
- **Bootkit / MBR** — infects the boot record, loading **before the OS**.
- **Hypervisor** — installs beneath the OS, turning it into a guest VM.

Because the compromised OS can't be trusted, detection needs **external/offline tools** — forensic boot media, memory analysis, or network-side detection — and removal often means **complete reimaging**. Rarely deployed in real pentests (stability + removal risk), but heavily tested.

> ◈ EXAM TELL
>
> "Most difficult to detect," "hides processes/files/connections from the OS," "intercepts system calls," or "requires reimaging to remove" ⇒ rootkit. The paired discriminator with #06: rootkit = the most **invisible implant**; valid credentials = the stealthiest because there's **no implant at all**. "Loads before the OS" ⇒ the bootkit/MBR sub-type specifically.

`kernel-level` `hides processes/files/conns` `intercepts system calls` `requires reimaging` `bootkit/MBR → before OS`

---

11 Browser extensions

An overlooked vector: the browser is always running, trusted, and sees credentials and web content. A malicious extension is persistent, high-value access that blends into normal use.

ACCESS & PERSISTENCE

Delivered by social engineering (a "useful" ad blocker or tool). Granted "read and change all your data on all websites," it can capture credentials from login forms, steal session cookies for hijacking, modify page content, exfiltrate data, redirect to phishing, and act on web apps as the user. It reloads every time the browser starts, so it **persists across sessions and reboots**, and it can maintain C2 over ordinary HTTPS that's indistinguishable from browsing. Hard to detect: runs inside a trusted process, normal-looking traffic, rarely audited. In enterprises, group policy / managed-browser config can push a malicious extension org-wide in one action.

> ◈ EXAM TELL
>
> Reach for browser extensions when a scenario stresses **ongoing credential/cookie capture that persists across reboots** while blending into HTTPS traffic. The delivery cue is "installed by the user" (social engineering); the scale cue is "deployed via group policy."

`runs in browser process` `captures form submissions` `cookie theft` `persists across reboots` `group-policy deployment`

---

12 Tampering with security controls

Not a standalone persistence method — an **enabler** that removes the guardrails (firewall, AV/EDR, logging, policy) so every other mechanism survives longer and stays undetected.

WHAT GETS TAMPERED

- **Firewalls:** Windows `netsh advfirewall set allprofiles state off` (or add-rule exceptions); Linux `iptables -F`, `ufw disable` — keeps network-based channels open.
- **AV / EDR:** `sc stop` / `net stop`, Defender via `Set-MpPreference -DisableRealtimeMonitoring $true` or the `DisableAntiSpyware` registry value — though modern EDR resists tampering.
- **Logging:** `wevtutil cl Security/System/Application`, PowerShell `Clear-EventLog`; Linux clears `/var/log/` and disables `auditd` — removes the evidence trail.
- **Policy:** weaken password/lockout policy, enable PowerShell execution — removes barriers.

◈ EXAM TELL

When a question asks how to make persistence **more durable / harder to detect** (rather than how to establish it), the answer is security-control tampering — it's the enabler, not the mechanism. Command cues: **wevtutil cl** / **Clear-EventLog** (log tampering), **netsh advfirewall** (firewall), disabling Defender (AV). A classic trap treats this as a standalone persistence technique — it isn't.

disabling Defender/AV | **wevtutil cl** / Clear-EventLog | **netsh advfirewall** | **iptables -F** | enables other persistence

CLEANUP

Re-enable every disabled control and restore modified policy to its documented original — a critical closeout step.

→ How the concepts correlate

The layered persistence model

Advanced access chains mechanisms so no single cleanup ends it:

- **C2 framework** — the operational platform.
- **Scheduled task** — relaunches the agent after termination.
- **Registry Run key** — launches the agent at logon.
- **Service** — boot-time persistence before any login.
- **New account** — backup path if the C2 is found.
- **Security-control tampering** — keeps AV/logging from exposing the above.
- **Valid credentials** — artifact-free final fallback.

Persistence mechanism ≠ access channel

- Scheduled task *(mechanism)* → reverse shell *(channel)*
- Service *(mechanism)* → C2 agent *(channel)*
- Registry key *(mechanism)* → any payload *(channel)*

CLEANUP CORRELATION

Every mechanism has a matching teardown: scheduled task → `schtasks /delete` · service → `sc delete` · new account → `net user … /delete` · registry → remove the value · web shell → delete the file · tampering → re-enable all controls.

Master keyword reference

When a stem drops one of these, jump straight to the concept.

KEYWORD / PHRASE	CONCEPT	CONTEXT
Survives reboot	Sched task / service / registry	Outlasts restarts (OS executes it)
Target connects to attacker	Reverse shell	Outbound direction
Attacker connects to target	Bind shell	Inbound direction
Behind NAT / firewall	Use reverse shell	Network constraint
LSASS / Mimikatz	Valid credentials	Credential dumping
Pass-the-hash	Valid credentials	Reuse without cracking
HKCU Run key	Registry	User-level, no admin
HKLM Run key	Registry	System-wide, admin required
Empire / Covenant / Mythic	C2 framework	Post-exploitation platform
In-memory execution	Empire / C2	Avoids disk detection
Kernel-level / hides processes	Rootkit	Most sophisticated
Requires reimaging	Rootkit removal	Severity indicator
HTTP file in web dir	Web shell	Web-based backdoor
Disguised as legit software	Trojan	Social-engineering delivery
Captures form submissions	Browser extension	Credential harvesting
Disabling Defender / AV	Control tampering	Enables other persistence
wevtutil cl / Clear-EventLog	Log tampering	Evidence removal
schtasks /create	Scheduled task	Windows command
crontab -e	Cron job	Linux command
sc create	Service creation	Windows service
net user /add	New account	Account persistence
Stealthiest persistence	Valid credentials	Looks like legit access
Most durable / invisible	Rootkit or valid creds	Context-dependent

Exam strategy & common traps

Scenario analysis framework

For any persistence question, run these six filters before reading the options:

1 · OS? Windows → schtasks, registry, services · Linux → cron, systemd, /etc/init.d

2 · Privilege? User → HKCU, user cron, user account · Admin → HKLM, services, SYSTEM tasks

3 · Network? Behind NAT/firewall → reverse shell · Direct access, can't host listener → bind shell

4 · Duration? Must survive reboot → service or scheduled task · Session-only → reverse shell alone

5 · Stealth? Max → valid credentials or rootkit · Moderate → C2 over HTTPS

6 · Establish or harden? Making existing persistence more durable → security-control tampering

Common traps

TRAP 1

Confusing **reverse** (target → attacker) with **bind** (attacker → target). This is the most-tested reversal.

TRAP 2

Forgetting **HKCU** needs only user privileges while **HKLM** needs admin.

TRAP 3

Treating a **rootkit** as just another tool rather than the most sophisticated, reimaging-required option.

TRAP 4

Forgetting **C2 frameworks** are platforms that combine persistence with operational capability across many hosts.

TRAP 5

Missing that **security-control tampering** is an enabler, not a standalone persistence mechanism.

◈ ONE MORE DISCRIMINATOR

"Stealthiest" vs "most invisible implant" split answers: **valid credentials** = stealthiest (no artifact), **rootkit** = most invisible implant (hides itself from the OS). If both appear as options, let the exact wording of the stem decide.

Study guide built from your Domain 5.1 source, mapped to PenTest+ PT0-003 Objective 5.1. Self-contained — no external assets, so it loads instantly and stays smooth on long scrolls. Cross-check the exact objective wording against CompTIA's current objectives PDF.